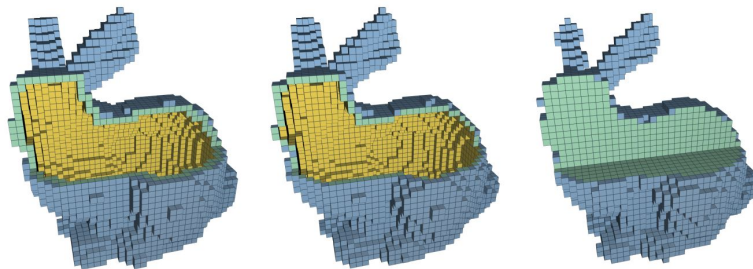


“

VoxelViz: Progressive Voxel-Based Visualization of 3D Models with Animated Build-Up



Vincent Jovian Christanto 黃財城 | Aryabima Mandala Putra 黃峰

Geometric Modelling 2025

Introduction

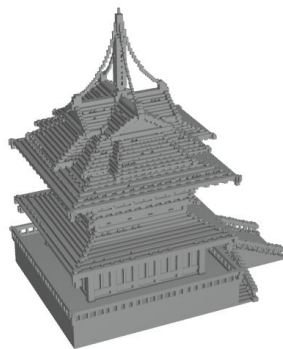
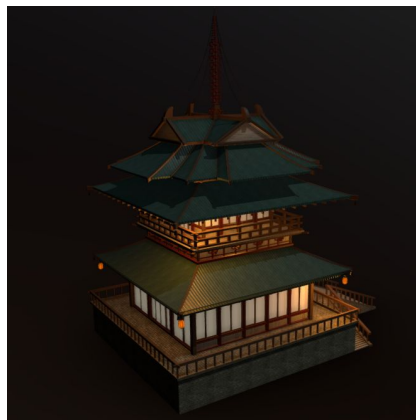
”

- **Voxel** : Volumetric Pixels
- Voxels **represent values** on a regular grid in three-dimensional space
- Typically **indicating the presence or absence of a part of a shape**
- Unlike polygons or meshes that represent just the surface of a model, **voxels can encode both surface and volumetric information**
- **Suitable for applications where internal structure matters**, such as medical imaging, simulations, or interactive educational tools.

What is VoxelViz?

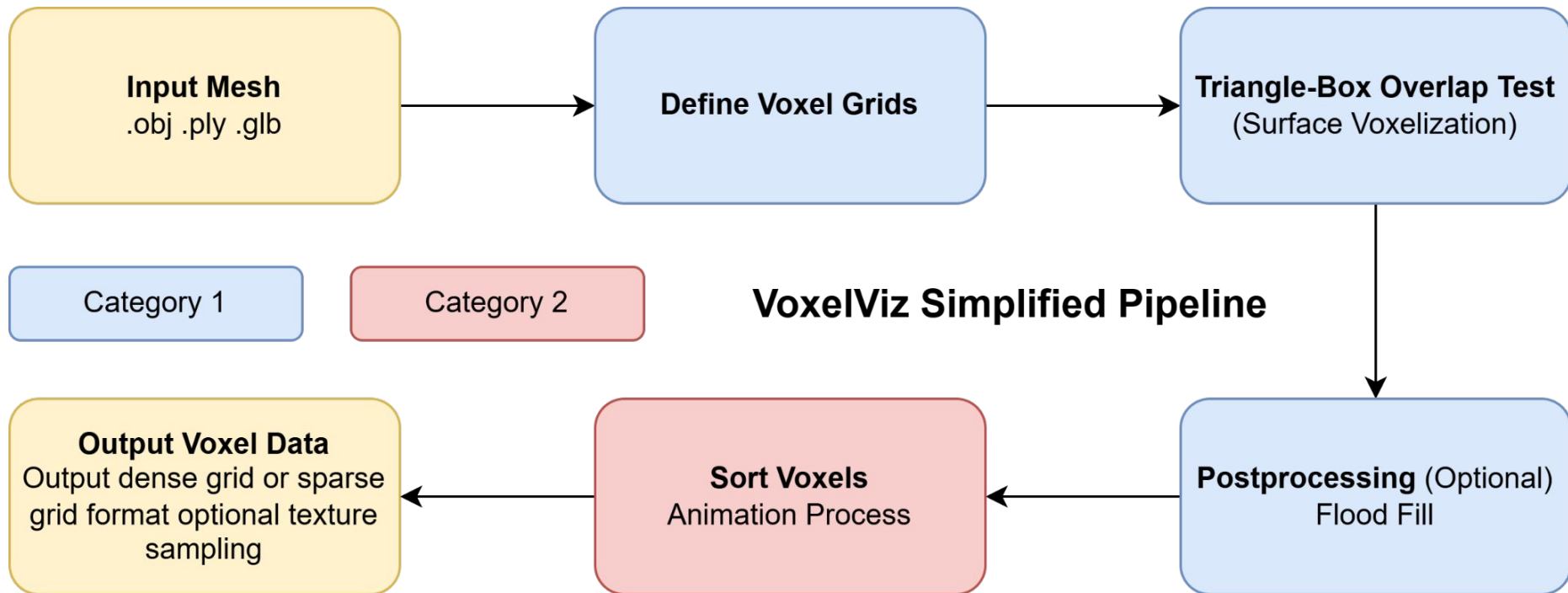
”

- Visualization tool that converts 3D models into voxels and animates their construction in real time
- It's designed primarily for educational and interactive purposes, helping users (especially students and beginners) better understand 3D shapes, spatial structures, and geometric composition by building up models voxel-by-voxel, like stacking digital LEGO blocks



How does it work?

”



Related Works

”

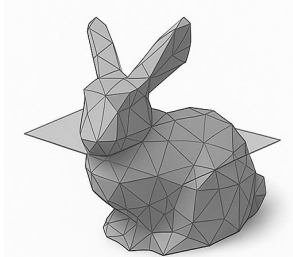
Voxelization Method :

- **Zhang et al. (2017)** : Scanline Voxelization
- **Schwarz & Seidel (2010)** : Triangle-Box Overlap Test, GPU Conservative Rasterization
- **Schwarz & Seidel, OpenVDB** : Octree-based Sparse Voxelization

Animation Method :

- **Li et al. (2014)**: Wavefront planning ensures structural connectivity
- **Hedayati et al. (2019)**: Discrete voxel sequencing (layer-wise or per-voxel insertion)
- **Pandey et al. (2006)**: Slicing procedures guide layer height and clarity

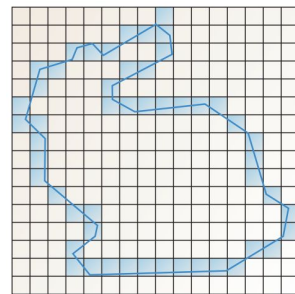
Voxelization Methods



”

- **Scanline Voxelization** *Zhang et al.*

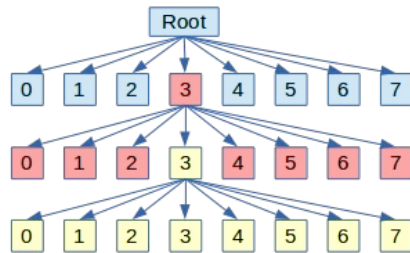
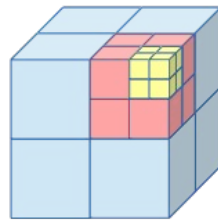
Concept : Scanline voxelization converts 3D triangles into voxels by slicing the model layer-by-layer, much like slicing bread. Each slice is a 2D cross-section (a scanline), and triangle intersections with that slice are used to fill voxel spans.



- **How It Works?**

- Choose a slicing axis (usually Z or Y)**
- For each slice (e.g., fixed Z-plane) :**
 - Find triangles that intersects this slice
 - Compute their intersection with the plane -> this forms 2D polygons
 - Use 2D scanline rasterization to fill voxel columns inside the polygons
- Repeat to all slices to complete the voxel grid -> $V[x][y][z_{current}] = \text{filled voxel}$**

Voxelization Methods



”

- **Octree-based Voxelization** *Schwarz & Seidel*

Concept : Instead of voxelizing into a full 3D grid (which is memory-intensive), the algorithm uses an octree: a tree structure that adapts resolution and only stores occupied space.

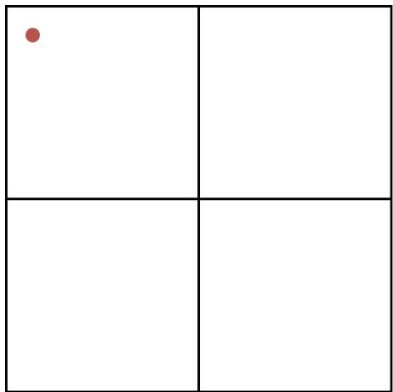
- **How It Works?**

- a. Start with the whole bounding box of the mesh as the root node.
- b. For each triangle in the mesh:
 - Test for intersection with the current octree node's cube.
 - If the triangle intersects and the node is too large:
 - Subdivide the cube into 8 children.
 - Repeat the test on each child.
 - If the node is small enough (leaf level) and intersected:
 - Mark it as a voxel.
- c. This continues recursively, adapting voxel size to model detail.

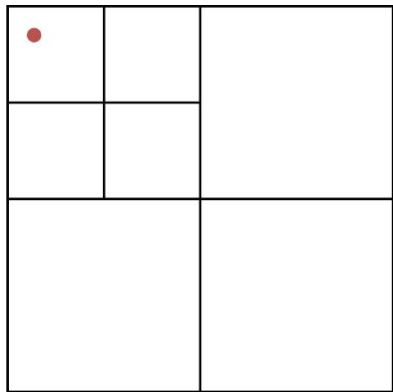
Voxelization Methods

”

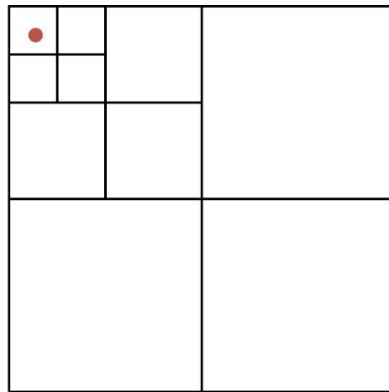
- Octree-based Voxelization



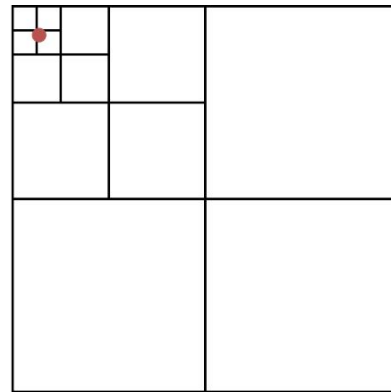
Root Node



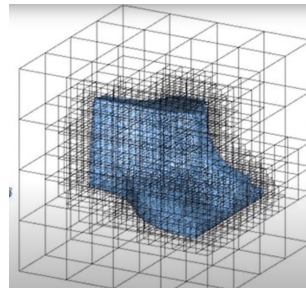
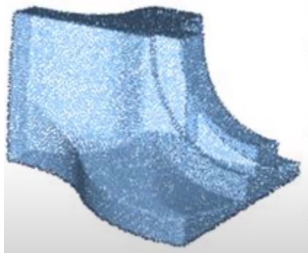
Level 1



Level 2



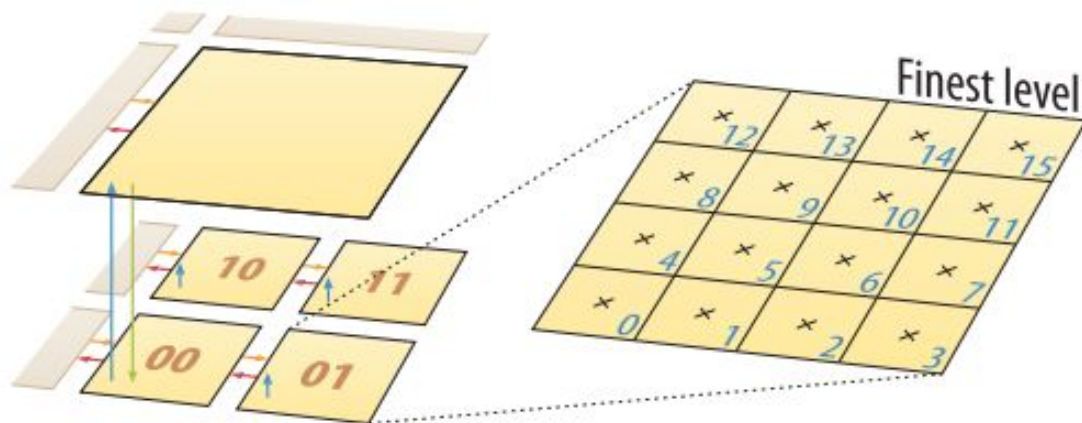
Level 3



Voxelization Methods

”

- Octree-based Voxelization



- +x neighbor node
- -x neighbor node
- First child node
- Parent node
- 01** Morton index
- 2** Sample index

Voxelization Methods

”

- **Triangle-box Overlap Test** *Zhang et al.*

Concept : For each triangle in the mesh, check which voxels it intersects using a fast triangle–box intersection test.

→ If the triangle touches a voxel → mark it as filled.

- **How It Works?**

- a. **Voxel Grid initialization.**

- Create a regular voxel grid over the Bunny’s bounding box.
 - Each voxel has a cube-shaped bounding box.

- b. **For each triangle in the mesh:**

- Compute its AABB (axis-aligned bounding box).
 - Identify all voxels it could possibly intersect.
 - For each candidate voxel:

- Perform triangle-box overlap test using Separating Axis Theorem (SAT)

- c. **If triangle intersects voxel -> Mark it filled.**

- d. **(Optional) Apply flood-fill for solid voxelization if volume is needed**

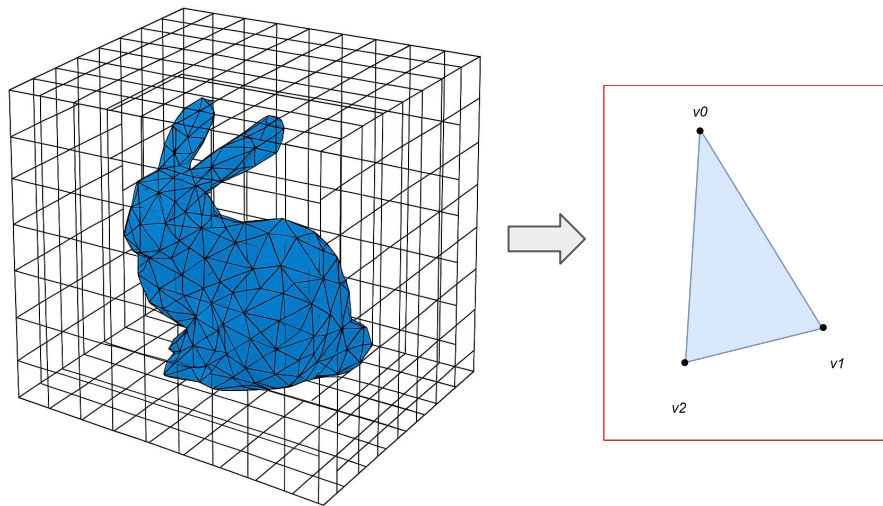
Voxelization Methods

”

- Triangle-box Overlap Test - *Computing AABB*

Goal :

Find which voxels a triangle might intersect, so we can loop only over a small block of voxels, not the whole grid



We got :

$$v0 = (x0, y0, z0)$$

$$v1 = (x1, y1, z1)$$

$$v2 = (x2, y2, z2)$$

We find triangle's bounding box:

$$\begin{aligned} i_{min} &= \lfloor \min(x0, x1, x2) \rfloor & i_{max} &= \lceil \max(x0, x1, x2) \rceil \\ j_{min} &= \lfloor \min(y0, y1, y2) \rfloor & j_{max} &= \lceil \max(y0, y1, y2) \rceil \\ k_{min} &= \lfloor \min(z0, z1, z2) \rfloor & k_{max} &= \lceil \max(z0, z1, z2) \rceil \end{aligned}$$

Voxelization Methods

”

- **Triangle-box Overlap Test - *Intersection Test***

Goal :

Check if the triangle's plane cuts through the voxel 3D cube. If not, skip this voxel entirely.

1. Compute triangle plane normal and offset :

$$\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0, \quad \mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$$

$$\mathbf{n} = \frac{\mathbf{e}_1 \times \mathbf{e}_2}{\|\mathbf{e}_1 \times \mathbf{e}_2\|}$$

$$\mathbf{n} \cdot \mathbf{x} + d = 0, \quad \text{where } d = -\mathbf{n} \cdot \mathbf{v}_0$$

2. Does this voxel intersect the triangle's plane?

For this, we only need to check two extreme corner of the voxel (farthest and closest in respect with the triangle's normal).

If all corners is on the same side of the plane then the plane misses the voxel entirely

Compute :

$$\begin{aligned} s_{min} &= \mathbf{n} \cdot \mathbf{v}_{min} + d \\ s_{max} &= \mathbf{n} \cdot \mathbf{v}_{max} + d \end{aligned}$$

Find so that $s_{min} \cdot s_{max} \leq 0$

$$n = (+, -, +)$$

Then:

- min corner = $(i, j + 1, k)$
- max corner = $(i + 1, j, k + 1)$

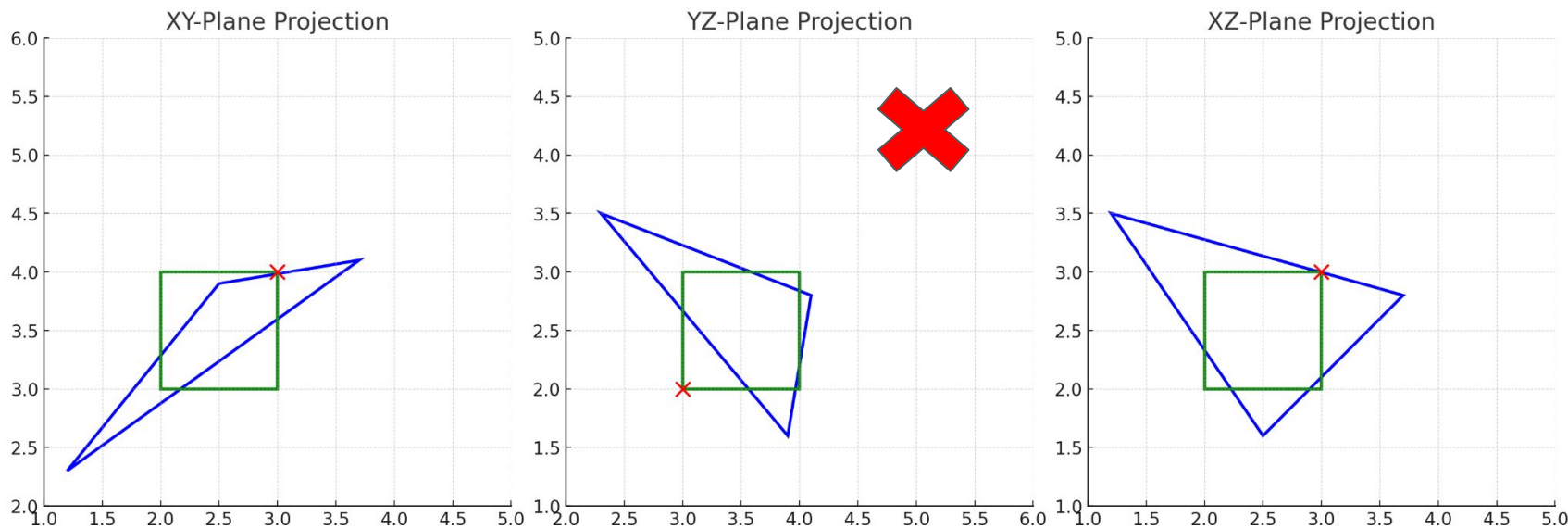
Voxelization Methods

”

- Triangle-box Overlap Test - *2D Edge Function Projection Tests (XY, YZ, XZ)*

Goal :

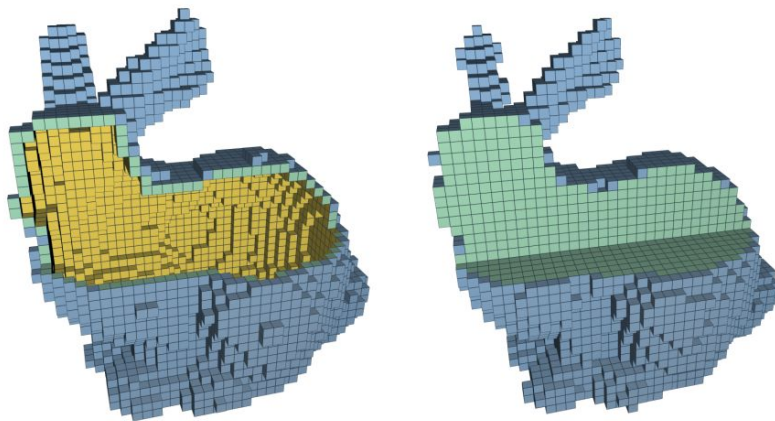
Only if all three projections return true, and the plane intersection test also passed, you mark this voxel as part of the surface



Solid Voxelization

”

- Use Flood-Fill Method



On a 3D Grid of voxels $V[x][y][z]$, assign labels :

- $V[x][y][z] = 0$ -> Empty
- $V[x][y][z] = 1$ -> Surface

Then BFS visit all empty and mark as “visited”. If empty & not visited then it must be interior. Mark $V[x][y][z] = 2$ -> interior, then just fill it.

Method Comparisons

”

Method	Accuracy	Orientation	Speed	GPU Ready	Pros	Cons
Triangle-Box Overlap	Supercover	Any	Very Fast	Yes	No holes, tunable thickness, GPU-parallel	Slight overfill
Scanline	Angle-sensitive	Axis Dependency	Fast	Yes	Simple, efficient for aligned shapes	Misses tilted features
Octree-Based Voxelization	Adaptive	Any	Fast	Yes	Huge memory savings, supports LOD, sparse grids	Harder to implement, not uniform grid

Implementation Test

”

Mesh Preparation & Ensuring Watertightness

- **Load the 3D Model**

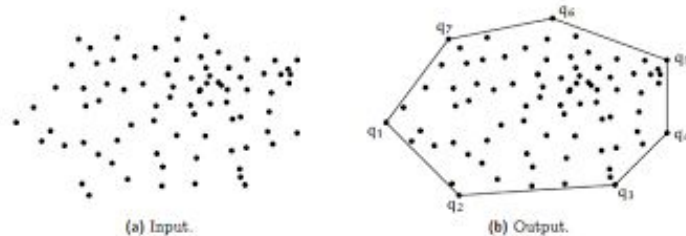
The first step is to import a 3D mesh (STL, OBJ, PLY, etc.) into our environment.

If the model consists of multiple parts, they are merged into a single continuous mesh.

- **Enforce Watertight Geometry**

Many voxelization algorithms, including the GPU-accelerated methods in Schwarz & Seidel (2010), require a watertight (closed) mesh to correctly classify “inside” versus “outside.”

If the imported mesh has small holes or gaps, we replace it with its convex hull. The convex hull is guaranteed to be watertight and preserves the overall envelope of the shape.



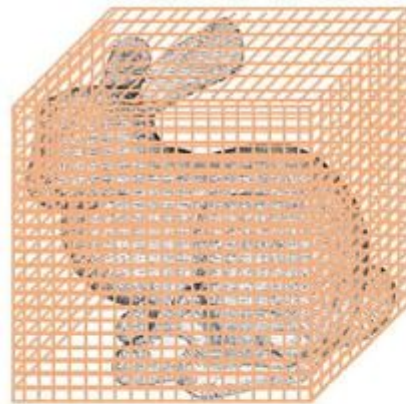
Implementation Test

Voxel Grid

”

Defining the Voxel Grid (Slice Thickness)

- **Compute the Axis-Aligned Bounding Box (AABB)**
- **Choose a Voxel Size**



The chosen “voxel size” just mean the size of each voxels. Smaller voxels yield higher fidelity but also increase computational load and total voxel count.

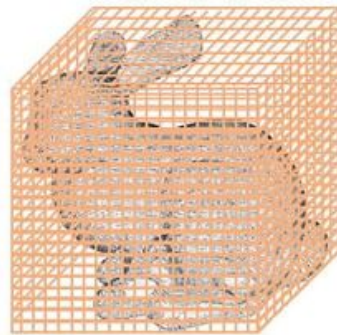
- **Establish Grid Dimensions (Cage)**

By dividing each dimension of the AABB by the voxel size (and rounding up), we compute how many voxels will fit along X, Y, and Z.

This results in a 3D grid of integer dimensions (nx, ny, nz).

Implementation Test

Voxel Grid



GPU-Accelerated Solid Voxelization (Schwarz & Seidel, 2010)

- **Invoke Voxelization**

`mesh.voxelized(pitch=voxel_size)`, which internally uses a GPU-accelerated “triangle-box overlap” approach (exactly as described in **Schwarz & Seidel, 2010** and covered before).

- **Create a Boolean Occupancy Grid**

The outcome is a 3D boolean array ($n_x \times n_y \times n_z$) where True indicates that the corresponding voxel lies within the original mesh volume. (True indicates inside of the mesh, while false means it's the background/empty) Additionally, the voxelization routine provides a 4×4 transform matrix that tells us the precise world-space location for the corner of the grid. Using this matrix, we can compute the exact center of each occupied voxel.

Implementation Test - Animation

”

Assigning Colors by Z-Layer (Pandey et al., 2006)

- **Select a Colormap**

Here we just made a simple color map, drawing inspiration from Pandey et al. (2006), who discuss color-coding slices to improve clarity, we sample a perceptually uniform colormap (e.g. “viridis” or “plasma”) for each horizontal layer.



Implementation Test - Animation

”

Determining Drop Sequence (Hedayati et al., 2019 & Li et al., 2014)

- **Extract All Filled Voxel Indices**

From the boolean occupancy grid, we list out the grid coordinates (ix, iy, iz) of every occupied voxel.

- **Choose an Ordering Strategy**

Simple Bottom-Up Sort: Sort all occupied voxels by ascending Z-index (iz). This ensures that cubes in lower layers always appear before cubes above them.

Reference: Hedayati et al. (2019) often group voxels by layer for robotic assembly. Our bottom-up method is a simplified version of their layered insertion.

Implementation Test - Animation

”

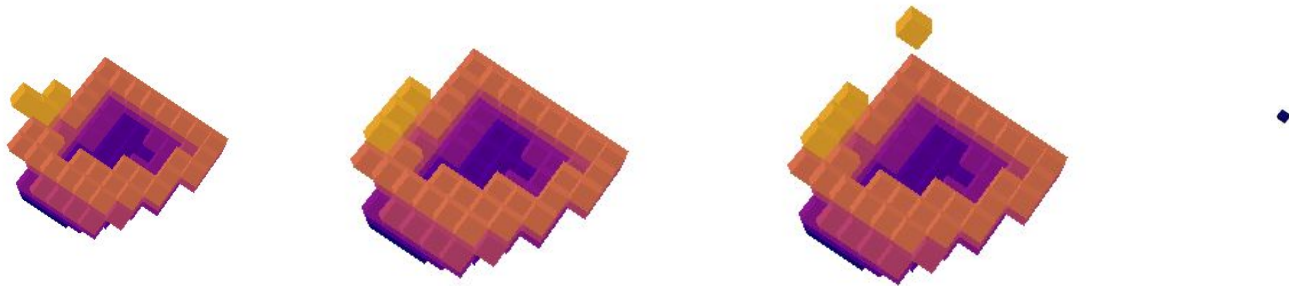
Animating Each Cube

- **Set a Drop Start Height**

We choose a start height a fixed offset above that top, then each voxel's animation begins from this “drop start” and ends at its own final center-height.

- **Frame-by-Frame Construction**

We build the animation frame by frame and construct it as a gif/mp4 file using ImageIO for gif and FFmpeg for an mp4 video.



Implementation Test - Animation

”

Here are some different ordering animations that we tested



Generative



Layer by Layer



Block by Block

Conclusion

”

VoxelViz combines three key advances:

Real-time Voxelization: Fast GPU methods allow on-the-fly conversion of 3D models into voxels.

Texture-Aware Coloring: Applying color schemes makes voxelized models visually informative

Progressive Animation: Stepwise reveals, by slice or individual block, help users understand 3D structure as it builds up.

By showing shapes assembling layer by layer or block by block, people can intuitively understand construction logic. Moving forward, we can make these capabilities in an easy-to-use interface for education & creativity purposes.

”

THANK YOU